COMSATS University Islamabad, Lahore Campus
Department of Computer Sciences

JUnit – MID Project - Semester Spring 2026

Course Title	Software Testing				Course Code	CSE482	Credit Hours: 3
Course Instructor	Ms. Yella Mehroze				Program Name	BSE	
Semester	8th	Batch:	FA22-BSE	Section	A ,B	Type	Lab
Total Marks		Obtained Marks:					
Name					Registration ID		

Question1:

Statement

[CLO <4>]; Implement automated test cases using a testing tool (JIRA, JUnit & Selenium).

Bloom Taxonomy Level<Applying>

Make use of selenium JUnit to perform the following tasks.

Task 1 –

Concept	JUnit Annotations (@Test, @BeforeEach, @AfterEach)
Project Area	Calculator utility module for portal helpers
Scenario	The development team wants to confirm that a simple calculator helper used inside the university portal behaves correctly and that a fresh object is prepared before each test.
Expected Result	The add and subtract methods return the correct values, and the test object is initialized and cleaned properly for every test run.
Files / Classes	RollNum-CalculatorService.java, RollNum-CalculatorServiceTest.java

Steps to perform:

1. Create a CalculatorService class with add and subtract methods.
2. Create a JUnit 5 test class.
3. Use @BeforeEach to initialize the object before every test.
4. Use @AfterEach to clean up after every test.
5. Use @Test to verify addition.
6. Use another @Test to verify subtraction.
7. Run the test class and confirm all tests pass.

Task 2 –

Concept	JUnit Assertions (assertEquals, assertTrue, assertFalse, assertThrows)
Project Area	Login and password validation rules
Scenario	The QA team needs to verify that email and password validation logic used by the portal correctly accepts valid values, rejects weak input, and throws an exception for invalid internal usage.
Expected Result	Valid email returns true, invalid password returns false, and null password handling throws the required exception.
Files / Classes	Two files, must have your names as class names as given in task 1

Steps to perform:

1. Create a LoginValidator class for email and password checks.
2. Write one test using assertTrue for a valid email.
3. Write one test using assertFalse for a weak password.
4. Write one test using assertThrows for null password input.
5. Run the test class.
6. Record the output and explain which assertion was used in each case.

Task 3 –

Concept	Advanced JUnit Lifecycle and Grouped Assertions (@BeforeAll, @BeforeEach, assertAll, assertNotNull, assertThrows)
Project Area	Result processing and grade summary module
Scenario	The release team wants confidence that a result-processing module calculates grades correctly, flags pass/fail status, builds a summary string, and rejects impossible marks.
Expected Result	Grade logic works correctly, grouped assertions pass together for valid data, and out-of-range values throw an exception.
Files / Classes	Two files, must have your names as class names as given in task 1

Steps to perform:

1. Create a ResultProcessor class to calculate grade, passing status, and summary.
2. Use @BeforeAll to print or prepare a shared start message.
3. Use @BeforeEach to create a fresh processor object.
4. Use assertAll to validate more than one outcome in a single test.
5. Use assertNotNull to verify that a summary is produced.
6. Use assertThrows to reject invalid marks such as 120.
7. Run the full test class and review the passed results.

Task 4 – Write on your own

Concept	Student choice (for example: boundary testing, null validation, business-rule testing, or string processing checks)
Project Area	Any Java module related to the student's FYP or any other functionality.
Scenario	Students must design one JUnit task on a project or website-related Java module of their own choice.
Expected Result	The selected feature should be tested with meaningful assertions and the expected output should be clearly justified.
Files / Classes	Student-defined Java class and JUnit test class

Steps to perform:

1. Choose one project or site of your choice.
2. Identify one Java logic area that can be unit tested.
3. Mention the concept name before the task.
4. Write a clear scenario.
5. Write step-by-step task instructions.
6. Write the expected result.
7. Implement the JUnit test case and capture output screenshots.

Student instruction: This task should not be limited to page opening. Test real logic such as registration validation, bill calculation, discount rules, marks processing, profile update validation, search filtering logic, or API response formatting logic.

Task 5 – Write on your own

Concept	Student choice (for example: exception testing, repeated tests, grouped assertions, or service validation)
Project Area	Any Java module related to the student's FYP or any other functionality.
Scenario	Students must design one additional JUnit task that covers a different logic area from Task 4.
Expected Result	The test should validate a different functionality and use suitable annotations and assertions for that case.
Files / Classes	Student-defined Java class and JUnit test class

Steps to perform:

1. Select a second functionality from your own project or chosen site.
2. Use a different concept from Task 4.
3. Write the scenario in the same format.

4. Provide clear steps and expected results.
5. Implement the JUnit test case.
6. Take screenshots and add them to the submission Word file.

Task 4, Task 5

- Students must design 2 additional JUnit tasks of their choice.
- Each additional task must clearly mention the concept, scenario and expected outcome.

Submission Requirements

- Java Project
- A word file containing screenshots of your scripts and outputs
- Do not copy commands from classmates or GPT blindly. You must have a clear understanding of each script line you have written. It will be evaluated in the VIVA.
- Copy the header format from this document and paste it in the word file you will submit
- Write your name and Id in that word file.
- Paste all your code in the word file (either make word tables or simply copy paste the classes with screenshot of outputs).
- Submit Java project, along with java files .

JUnit is used for unit testing Java code, not for browser automation like we did in selenium webdriver. Therefore, the student-designed tasks should focus on logic, validation, services, utilities, or backend-related features of the selected project or site.